

FitLook Final Security Remediation Report

Project: FitLook
Report date: 16 August 2026
Assessment type: Source-code security review and local remediation verification
Status: Remediation pass completed

1. Executive summary

The original review identified two critical, three high, three medium, and one low-severity security findings. The remediation pass addressed the immediately exploitable application weaknesses, introduced reusable security controls, upgraded vulnerable production dependencies, and added regression tests for the highest-risk logic.

The most important changes prevent users from bypassing AI token charges in production, stop server-side requests to internal/private networks, protect private user media with ownership checks, reject active or spoofed profile-image uploads, limit abusive request volumes, and apply baseline browser and transport security headers.

Local verification completed successfully:

Verification	Result
Security regression tests	3 passed, 0 failed
User application production build	Passed
Administrator application production build	Passed
Production dependency audit	0 known vulnerabilities
Patched Sharp version	0.35.3

2. Remediation overview

Finding	Original severity	Remediation status
User-controlled development mode bypassed token billing	Critical	Fixed
Authenticated SSRF through external image URLs	Critical	Fixed
Private user media served without access control	High	Fixed for local user-media paths
Upload validation trusted MIME type and unsafe extensions	High	Fixed for public profile uploads; tightened for product uploads
Vulnerable production dependencies	High	Fixed
Missing authentication and expensive-operation rate limits	Medium	Fixed
Persistent browser storage of bearer/admin credentials	Medium	Hardened to session-scoped storage

Shared administrator secret lacked modern identity controls

Medium

Partially hardened; identity-bound admin authentication remains recommended

Missing HTTP security headers and verbose health response

Low

Fixed

3. Fixes performed

3.1 Production billing bypass prevention

The public signup route no longer accepts a user-supplied development-mode value. New users are always created with development mode disabled.

The development-mode endpoint now returns `404` unless both of these server-controlled conditions are true:

- The environment is not production.
- `ENABLE_DEV_MODE=true` is explicitly configured.

Token and video-cost calculations now use a central `developmentBillingBypass()` function. The function always returns false in production, even if a database record contains `devMode: true`.

Security result: users cannot enable free AI generation in production by manipulating signup or profile requests.

3.2 SSRF and outbound request protection

A centralized safe outbound-fetch implementation was added. User-controlled remote requests now enforce:

- HTTPS only.
- No embedded URL credentials.
- Port 443 only.
- DNS resolution before connecting.
- Rejection of loopback, private, link-local, multicast, reserved, and cloud-metadata address ranges.
- Validation after every redirect.
- A maximum of three redirects.
- A configurable request timeout, defaulting to 12 seconds.
- Streamed response-size enforcement instead of unbounded buffering.

The control was connected to external try-on image fetching and product preview/import fetching.

Security result: authenticated users can no longer use these routes to probe common internal networks or download unlimited-size responses into server memory.

3.3 Private-media authorization

The uploads route now distinguishes public catalog media from private user media. Private paths require a valid signed user token and an ownership match.

Authorization permits access only when:

- The requested media path is under that user's own `/uploads/users/{userId}/` namespace; or
- The requested root-level profile image exactly matches the authenticated user's stored profile-photo path.

Private responses use `Cache-Control: private, no-store`. The client attaches the media credential only to `/uploads/` URLs returned by authenticated API calls.

Security result: one user cannot retrieve another user's private try-on, closet, or profile media using only its path.

3.4 Upload hardening

Profile uploads now accept only the required raster types: JPEG, PNG, WebP, AVIF, HEIC, and HEIF. SVG and arbitrary `image/*` content are no longer accepted.

Accepted profile files are decoded and re-encoded as JPEG. Processing applies:

- A 40-megapixel input-pixel limit.
- Automatic orientation correction.
- Server-generated JPEG output.
- Deletion of the original uploaded file after successful conversion.
- Cleanup of failed conversion output.

Product upload MIME types were restricted to JPEG, PNG, WebP, and AVIF.

Static files receive `X-Content-Type-Options: nosniff`; unexpected extensions are served as attachments instead of inline content.

Security result: active SVG payloads and straightforward MIME/extension spoofing can no longer reach the public profile-upload path unchanged.

3.5 Request-abuse controls

Reusable in-process rate limiting was added with standard response headers and `Retry-After` behavior. Limits now cover:

- All API traffic.
- Signup attempts.
- Login attempts keyed by IP and account identifier.
- Username availability probing.
- Closet operations.
- AI try-on and video-generation operations.

Security result: brute-force attempts, username probing, upload floods, and rapid billable-provider requests are constrained.

For horizontally scaled production deployments, Redis-backed distributed rate limiting is still recommended so limits are shared by every application instance.

3.6 Browser and HTTP hardening

The server now:

- Disables the `X-Powered-By` framework header.
- Enables a Content Security Policy.
- Enables HSTS in production.
- Denies framing with CSP and `X-Frame-Options`.
- Disables MIME sniffing.
- Applies a strict referrer policy.
- Restricts browser permissions such as camera, microphone, and geolocation.
- Enables same-origin opener isolation.
- Returns only a minimal health result instead of exposing MongoDB state separately.

User JWTs and the administrator credential were moved from persistent `localStorage` to tab/session-scoped `sessionStorage`.

Security result: the browser attack surface, credential persistence window, clickjacking exposure, and server fingerprinting are reduced.

3.7 Authentication hardening

New passwords must contain at least 12 characters. Existing bcrypt hashing remains configured at cost 12. Login messages continue to avoid revealing whether an email or username exists.

Security result: newly created password credentials have a stronger minimum baseline, while login enumeration remains constrained.

3.8 Dependency remediation

The direct Sharp dependency was upgraded from the vulnerable 0.33 line to version 0.35.3. The package lock was refreshed and vulnerable transitive packages were updated or removed.

Final production audit result:

```
Critical: 0
High:    0
Moderate: 0
Low:     0
Total:   0
```

3.9 Security regression tests

New tests verify that:

- Production can never activate the development billing bypass.
- Development bypass requires both server opt-in and the user's development flag.
- SSRF classification rejects loopback, RFC1918, link-local, metadata, and private IPv6 addresses while permitting representative public addresses.

The root project now includes an explicit `npm test` command scoped to its own security test directory.

4. Files changed

File	Purpose
<code>server/utils/security.js</code>	Central headers, rate limiting, SSRF-safe fetch, media authorization, and production billing gate
<code>server/index.js</code>	Mounts global security controls, protected media serving, and hardened health response
<code>server/routes/auth.js</code>	Signup/login limits, password policy, upload re-encoding, and development-mode restrictions
<code>server/routes/tryons.js</code>	Production-safe billing logic, AI rate limiting, and safe remote-image fetching
<code>server/routes/closet.js</code>	Production-safe billing logic and operation rate limiting
<code>server/routes/products.js</code>	Safe product-page fetching and stricter product image types
<code>src/App.jsx</code>	Session-scoped user credential handling and protected media URLs
<code>admin/src/AdminApp.jsx</code>	Session-scoped administrator credential handling
<code>tests/security.test.js</code>	Security regression coverage
<code>package.json</code> / <code>package-lock.json</code>	Test command and patched dependency versions
<code>SECURITY_ASSESSMENT.md</code>	Original findings plus remediation status

5. Verification evidence

The following commands completed successfully after remediation:

```
npm test
npm run build
npm run admin:build
npm audit --omit=dev --json
```

Results:

- Three security tests passed.
- Both Vite production bundles compiled successfully.
- The production dependency audit returned an empty vulnerability set.
- Syntax validation passed for the changed backend modules.

6. Remaining deployment actions

These actions require deployment infrastructure or organizational decisions and cannot be completed solely in the local source tree:

1. Rotate JWT, administrator, database, payment, AI-provider, storage, and Redis credentials before deployment.
2. Replace the shared administrator secret with identity-bound administrator accounts, MFA, role-based authorization, short-lived sessions, revocation, and administrator audit logs.
3. Use HttpOnly, Secure, SameSite cookies with CSRF protection where the deployment architecture supports them. The implemented session storage is safer than persistent local storage but remains readable by same-origin JavaScript.
4. Replace media credentials in query parameters with short-lived purpose-bound signed media URLs or authenticated private object-storage delivery.
5. Use Redis-backed distributed rate limits in multi-instance production.
6. Place private media in private object storage and apply retention, deletion, encryption, and access-audit policies.
7. Perform an authorized staging penetration test against the deployed application, MongoDB authorization, PhonePe callbacks, storage/CDN behavior, proxy configuration, and AI-provider integrations.
8. Add continuous dependency, secret, SAST, and dynamic security scanning to CI/CD.

7. Final assessment

The critical source-level exploitation paths identified in the original assessment have been remediated and protected by regression tests. Local builds and dependency checks are clean. The application has a substantially improved baseline, but production approval should remain conditional on completing the identity, secret-rotation, distributed-rate-limit, private-storage, and staging-validation actions listed above.

This report documents a local source-code remediation pass. It is not a certification of the security of any deployed environment or third-party integration.